

Documentación de Arquitectura Técnica Implementada

Sistema Integrador de Reservas para Community Tours Sian Ka'an

Revisión técnica: 15 de septiembre de 2026 | Versión documental: v2

1. Introducción

Este documento describe la arquitectura técnica realmente implementada al 15 de septiembre de 2026. La fuente de verdad es el código del repositorio: backend, frontend, Electron, PostgreSQL, package.json y migraciones SQL.

2. Objetivo

Servir como checkpoint técnico actualizado para continuar el cierre del sistema sin depender de documentos históricos que ya fueron superados por la implementación.

3. Arquitectura General

La aplicación conserva una arquitectura por capas: Electron contiene la experiencia de escritorio, el frontend vanilla consume la API REST, Express centraliza reglas y PostgreSQL persiste los datos. El frontend nunca accede directamente a PostgreSQL.

Capa	Implementación	Responsabilidad
Electron	electron/main.js y electron/preload.js	Contenedor de escritorio, ciclo de vida de ventana y exportación PDF del Daily.
Frontend	frontend/index.html, CSS y JavaScript modular	Pantallas, navegación hash, consumo de API y renderizado operativo.
API	Node.js + Express	Rutas REST, autenticación, autorización, validación y respuestas HTTP.
Servicios	backend/services	Reglas de negocio, transacciones, capacidad y auditoría.
Datos	PostgreSQL mediante backend/config/database.js	Persistencia relacional con consultas parametrizadas.

4. Electron

- El punto de entrada declarado en package.json es electron/main.js.
- Electron abre la aplicación local y expone funciones controladas mediante preload.
- La exportación PDF del Daily se realiza desde el proceso principal con el canal daily:export-pdf.
- El build Windows está configurado con electron-builder; el release final debe reconstruirse después de pruebas reales.

5. Frontend

- El frontend usa HTML, CSS y JavaScript sin framework.
- La navegación usa rutas hash como #/dashboard, #/reservaciones, #/operaciones, #/daily, #/historial y #/bitacora.
- Todas las operaciones de datos usan fetch contra /api.
- Las pantallas implementadas cubren Login, Dashboard, Reservaciones, Operaciones, Daily, Historial, Bitácora, Usuarios y Configuración.

6. API REST

Ruta	Uso principal	Acceso
/api/auth	Login	Público para login
/api/dashboard	Resumen operativo real	Usuarios autenticados
/api/reservaciones	Listado, creación, edición, cancelación y asignación a transporte	Consulta lee; Administrador modifica
/api/operaciones	Operaciones, detalle y sugerencias por fecha	Consulta lee; Administrador modifica
/api/transportes	Transportes por operación	Consulta lee; Administrador modifica
/api/daily	Daily operativo y observaciones por fecha	Usuarios autenticados
/api/bitacora	Auditoría consultable	Administrador
/api/usuarios	Usuarios y roles	Administrador
/api/tours, /api/paises, /api/plataformas	Catálogos base	Según ruta y rol
/api/guias, /api/operadores, /api/vehiculos	Catálogos operativos	Según ruta y rol

7. PostgreSQL

Tabla	Propósito
roles	Catálogo de permisos funcionales.
usuarios	Cuentas de acceso con rol, correo, contraseña hasheada y estado.
tours	Catálogo de tours vendibles.
paises	Catálogo de nacionalidades o países del cliente.
plataformas	Origen comercial de la reservación.
vehiculos	Unidades disponibles, placas, color, capacidad y estado.
operadores	Conductores u operadores de transporte.
guias	Guías asignables a grupos operativos.
operaciones_tour	Grupo operativo por fecha, tour, turno y número de grupo.
transportes_operacion	Transporte asociado a un grupo operativo.
reservaciones	Datos comerciales, pickup, turno, estado y asignación operativa.
daily_observaciones	Observaciones generales por fecha del Daily.
bitacora	Registro de auditoría asociado a usuario y reservación.

8. Autenticación y Autorización

- El login se realiza mediante POST /api/auth/login.
- Las contraseñas se validan con bcryptjs.
- La sesión de API usa JWT.
- La autorización por rol se valida en middleware backend.
- El rol Administrador modifica información; el rol Consulta conserva acceso de lectura según módulo.

9. Reservaciones

- Las nuevas reservaciones se registran como Pendiente.
- Los estados documentados son Pendiente, Confirmada, Activa, Cancelada y Completada.
- El turno es explícito: Mañana o Tarde.
- Puede existir turno NULL solo para datos heredados.
- No se documentan transiciones automáticas de estado.
- pickup_time pertenece a cada reservación.
- La asignación a transporte es manual y valida fecha, tour, turno y capacidad.

10. Operaciones y Grupos

Relación o restricción	Regla vigente
roles 1:N usuarios	Cada usuario pertenece a un rol.
tours 1:N reservaciones	Cada reservación corresponde a un tour.
tours 1:N operaciones_tour	Cada grupo operativo corresponde a un tour.
guias 1:N operaciones_tour	Un guía puede aparecer en varios grupos.
operaciones_tour 0:1 transportes_operacion	Cada grupo puede tener como máximo un transporte.
vehiculos 1:N transportes_operacion	Un vehículo puede usarse en distintos grupos.
operadores 1:N transportes_operacion	Un operador puede usarse en distintos grupos.
transportes_operacion 1:N reservaciones	Varias reservaciones pueden asignarse al transporte del grupo.
reservaciones 1:N bitacora	Cada cambio relevante de reservación queda trazado.
fecha + tour + turno + grupo	Identidad única de operación.

- Una operación representa un grupo.
- La identidad lógica de una operación es fecha + tour + turno + numero_grupo.
- Los grupos permitidos son 1 y 2.
- Cada grupo admite hasta 12 PAX activos.
- Un tour en una fecha y turno admite hasta 24 PAX activos.
- hora_inicio representa la primera hora de pickup del grupo.

11. Transporte, Guía y Operador

- Cada grupo puede tener un guía asignado.
- Cada grupo puede tener como máximo un transporte.
- El transporte puede asociar vehículo, operador y observaciones del operador.
- Un transporte con vehículo debe respetar reglas de capacidad y mínimo operativo cuando ya tiene reservas activas.

12. Operaciones Sugeridas

Caso	Resultado
Agrupación	Fecha + tour + turno.
Estados incluidos	Pendiente, Confirmada y Activa.
Estados excluidos	Cancelada y Completada.
1 a 12 PAX	Sugiere 1 grupo.
13 a 24 PAX	Sugiere 2 grupos.
>24 PAX	Mantiene máximo 2 grupos y reporta excedente.
Asignación	No distribuye clientes automáticamente; el usuario asigna según hotel, ruta y pickup.

13. Daily

El Daily operativo muestra salidas por fecha con Tour, Turno, Grupo, Fecha, Primera hora de pickup, Guía, Vehículo, Operador y PAX. Las reservaciones se ordenan por pickup_time.

- GET /api/daily/operativo entrega operaciones, transportes y reservaciones sin asignar.
- GET/PUT /api/daily/observaciones administra observaciones por fecha.
- La impresión y PDF generan hoja por grupo/transporte cuando corresponde.

14. Dashboard

Métrica	Fuente
Reservaciones vigentes de hoy	GET /api/dashboard/resumen
PAX vigentes	GET /api/dashboard/resumen
Grupos preparados	GET /api/dashboard/resumen
Grupos pendientes	GET /api/dashboard/resumen
Reservaciones sin turno	GET /api/dashboard/resumen
Alertas de capacidad	GET /api/dashboard/resumen
Últimas 10 reservaciones	GET /api/dashboard/resumen
Salidas de hoy	GET /api/dashboard/resumen

15. Historial y Bitácora

- Historial consulta reservaciones con filtros desde /api/reservaciones.
- Bitácora se consulta desde /api/bitacora y está protegida para Administrador.
- Crear, modificar, cancelar y asignar transportes registra auditoría cuando aplica.

16. Seguridad

- No se exponen credenciales PostgreSQL al frontend.
- Las consultas SQL usan parámetros.
- Las contraseñas se almacenan hasheadas.

- Los permisos se validan en backend.
- Los secretos deben vivir en .env.

17. Configuración Runtime

- La configuración sensible se toma de variables de entorno.
- La conexión PostgreSQL se centraliza en backend/config/database.js.
- El backend Express sirve el frontend en /app.
- package.json define start, desktop, pack:win y dist:win.

18. Installer

Existe configuración técnica de empaquetado con electron-builder. El release final queda pendiente de reconstrucción después del cierre documental, datos reales y pruebas de aceptación.

19. Integraciones Pendientes

FareHarbor, GetYourGuide y WhatsApp Business están contempladas como plataformas u origen de reservación, pero no existe sincronización automática implementada ni contratos técnicos autorizados en el código.

20. Estado Técnico Actual

Área	Estado	Observación
Backend	Implementado	API REST, servicios, validadores, seguridad y auditoría.
Frontend	Implementado	Pantallas operativas en JavaScript vanilla.
Base de datos	Implementada	Schema actual más migraciones 001 a 007.
Daily y Dashboard	Implementados	Ambos consumen datos reales del backend.
Integraciones	Pendiente	Requieren documentación y acceso externo.
Pruebas reales	Pendiente	Faltan datos reales, aceptación de usuarios e installer final.

Anexo A. Comandos de Desarrollo

Comando	Descripción
npm start	Inicia el backend Express y sirve el frontend en /app.
npm run desktop	Inicia Electron sobre el backend local.
npm run pack:win	Genera paquete Windows sin instalador final.
npm run dist:win	Genera instalador Windows con electron-builder.
python scripts/generar_documento_arquitectura_implementada.py	Genera este documento en versión nueva.